

Posto: Probabilistic Safety Monitoring

User Guide

1 What Posto Does

Posto performs probabilistic safety monitoring by combining simulation, uncertain log generation, and safety verification.

1. Simulating trajectories

- **Equation mode:** next state computed from analytic expressions in a JSON file.
- **ANN mode:** next state produced by a trained neural network model (.h5).

2. Generating uncertain logs

- A single reference trajectory is simulated.
- Logged values are replaced with intervals of fixed half-width `--dtlog`.
- A random subset of time steps is logged (probability `--prob`), with $t = 0$ always logged.

3. Checking safety

- Safety constraints come from a JSON model file.
- Checks both the log samples and trajectories consistent with the log.
- Outputs SAFE / UNSAFE along with visualizations.

2 Installation & Environment

2.1 Python Dependencies

Install required libraries:

```
pip install numpy matplotlib docopt tensorflow
```

The system attempts:

```
from tensorflow.keras.models import load_model
# fallback:
from keras.models import load_model
```

2.2 Environment Variable: POSTO_ROOT_DIR

Set it before running Posto:

```
export POSTO_ROOT_DIR=/absolute/path/to/Posto
```

To make this permanent, add it to `/.bashrc`.

3 Repository Layout

```
Posto/  
  posto.py  
  System.py  
  Parameters.py  
  artEval.py  
  artEvalNN.py  
  art/  
  lib/  
    Equation.py  
    ANN.py  
    GenLog.py  
    TrajSafety.py  
    TrajValidity.py  
    JFBF.py  
    Visualize.py  
  models/  
  logs/  
  dev/  
    ModelANN.py  
    Model.py
```

4 Running the Posto Artifact (OVA)

1. Install Oracle VirtualBox (version 7.0+) and the matching Extension Pack.
2. Download and unzip the artifact.
3. Open VirtualBox → **File** → **Import Appliance**.
4. Select the provided .ova file.
5. Click **Next** and then **Import**.
6. Start the VM.

Posto Location Inside the VM

```
~/Desktop/Posto  
cd ~/Desktop/Posto
```

5 Model JSON Format

Each model JSON must include:

- `state_vars`
- `constants` (optional)
- `ranges` (optional)
- `equations` (required for equation mode)
- `safety_constraints`

5.1 Example Model

```
{
  "state_vars": ["x", "y"],
  "constants": {"dt": 0.01},
  "ranges": {"ep": [-0.002, 0.002]},
  "equations": {
    "x'": "x + dt * (-y - (1.5*(x*x) - 0.5*(x*x*x) - 0.5)) + ep",
    "y'": "y + dt * ((3*x) - y) + ep"
  },
  "safety_constraints": [
    { "state": "x", "op": "le", "const": -0.10 },
    { "state": "y", "op": "le", "const": -0.10 }
  ]
}
```

5.2 Safety Constraint Semantics

Unsafe whenever constraint evaluates to true:

- "ge": $\text{state} \geq c$
- "le": $\text{state} \leq c$
- "gt": $\text{state} > c$
- "lt": $\text{state} < c$

6 Command-Line Interface

6.1 Available Commands

```
postto.py behavior
postto.py generateLog
postto.py checkSafety
```

6.2 Global Arguments

- --log=<directory or logfile>
- --mode=<equation | ann>
- --model_path=<path>
- --states=<comma-list> (ANN mode)
- --constraints=<json> (ANN safety)

6.3 Command Descriptions

behavior Simulates trajectories and outputs plots in `logdir/img/`.

generateLog Creates an interval-valued log file and uncertainty plots.

checkSafety Evaluates safety on log samples and consistent trajectories using the Jeffreys Bayes Factor.

7 Custom / Developer Mode

`dev/Model.py` allows implementing custom next-state functions.

7.1 Minimal Custom Model

```
def my_getNextState(state):
    x, y = state
    x = x + 0.1 + random.uniform(-0.01, 0.01)
    y = y - 0.05 + random.uniform(-0.01, 0.01)
    return (x, y)
```

7.2 Injecting Custom Dynamics into System

```
sys = System(log_path="logs/custom.lg", mode=None, model_path=None)
sys.getNextState = my_getNextState

init_box = [[0.0,1.0], [0.0,2.0]]
sys.generateLog(init_box, T=100, prob=50, dtlog=0.01)
```

7.3 Example With Three States

```
def my_getNext(state):
    x, y, z = state
    # Custom dynamics here
    return (x_next, y_next, z_next)
```

```
sys = System(log_path="logs/custom.lg",
             states=['x','y','z'])

sys.getNextState = my_getNext

sys.behaviour(init_box, T=1000)
sys.generateLog(init_box, T=2000, prob=2, dtlog=0.02)
sys.checkSafety()
```